

Hands-On Lab: Kubernetes Storage (Volumes, PV, PVC, StorageClass) using **kops**

This lab walks you step-by-step through **real storage scenarios** in a Kubernetes cluster created with **kops** (typically on AWS using EBS).

You'll learn:

1. Pod Volumes (**emptyDir**)
 2. Static Provisioning (PV + PVC)
 3. Dynamic Provisioning (StorageClass)
 4. Reclaim Policies
 5. Volume Binding Modes
 6. Cleanup & Verification
-

Lab Prerequisites

- Working Kubernetes cluster created via **kops**
- **kubect**l configured
- AWS permissions (if using EBS dynamic provisioning)

Verify:

```
kubectl get nodes
```

```
kubectl get storageclass
```

You should see a default StorageClass (e.g., gp2/gp3).

Lab 1 – Understanding Basic Pod Volume (**emptyDir**)

Goal

Understand ephemeral volume behavior.

Step 1 – Create Pod with emptyDir

```
apiVersion: v1
kind: Pod
metadata:
  name: emptydir-pod
spec:
  containers:
  - name: app
    image: nginx
    volumeMounts:
    - mountPath: /data
      name: cache
  volumes:
  - name: cache
    emptyDir: {}
```

Apply:

```
kubectl apply -f emptydir.yaml
```

Step 2 – Write Data

```
kubectl exec -it emptydir-pod -- /bin/bash
echo "Hello K8s Storage" > /data/test.txt
exit
```

Verify:

```
kubectl exec -it emptydir-pod -- cat /data/test.txt
```

Step 3 – Delete Pod

kubectrl delete pod emptydir-pod

Recreate and check file again → ❌ File is gone.

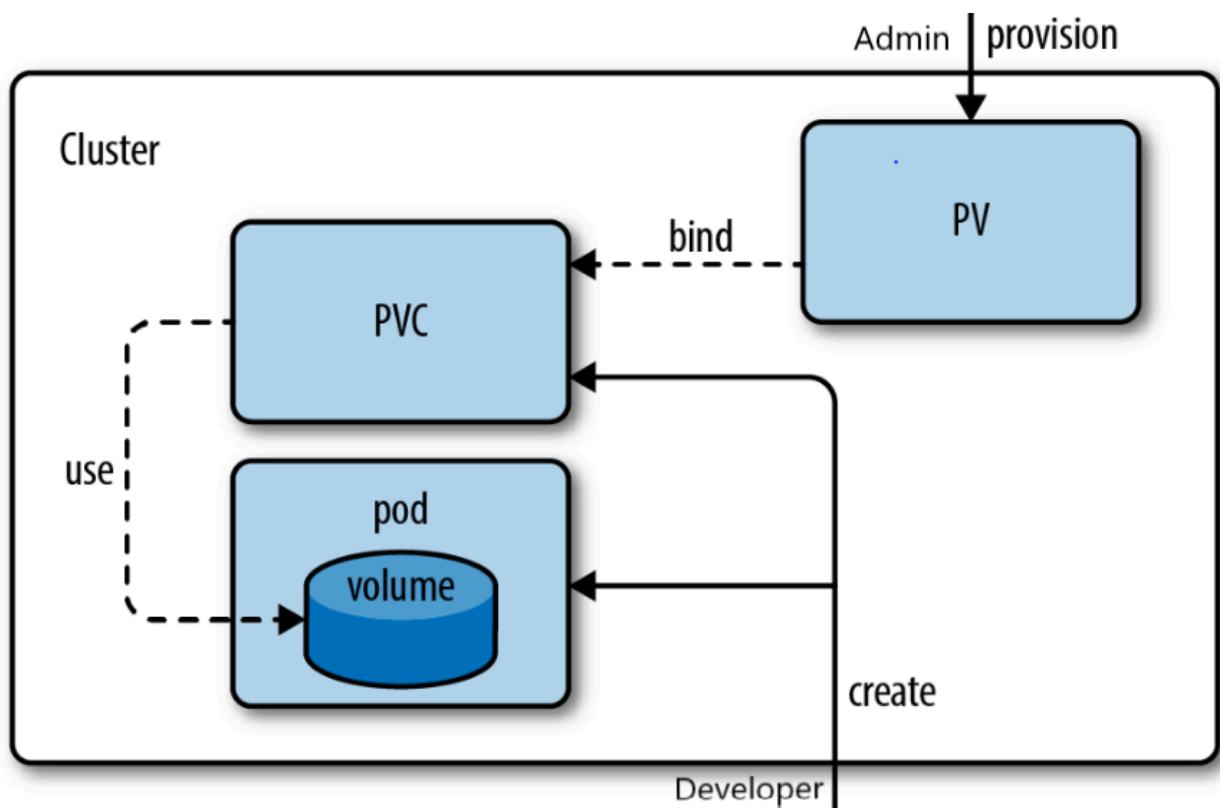
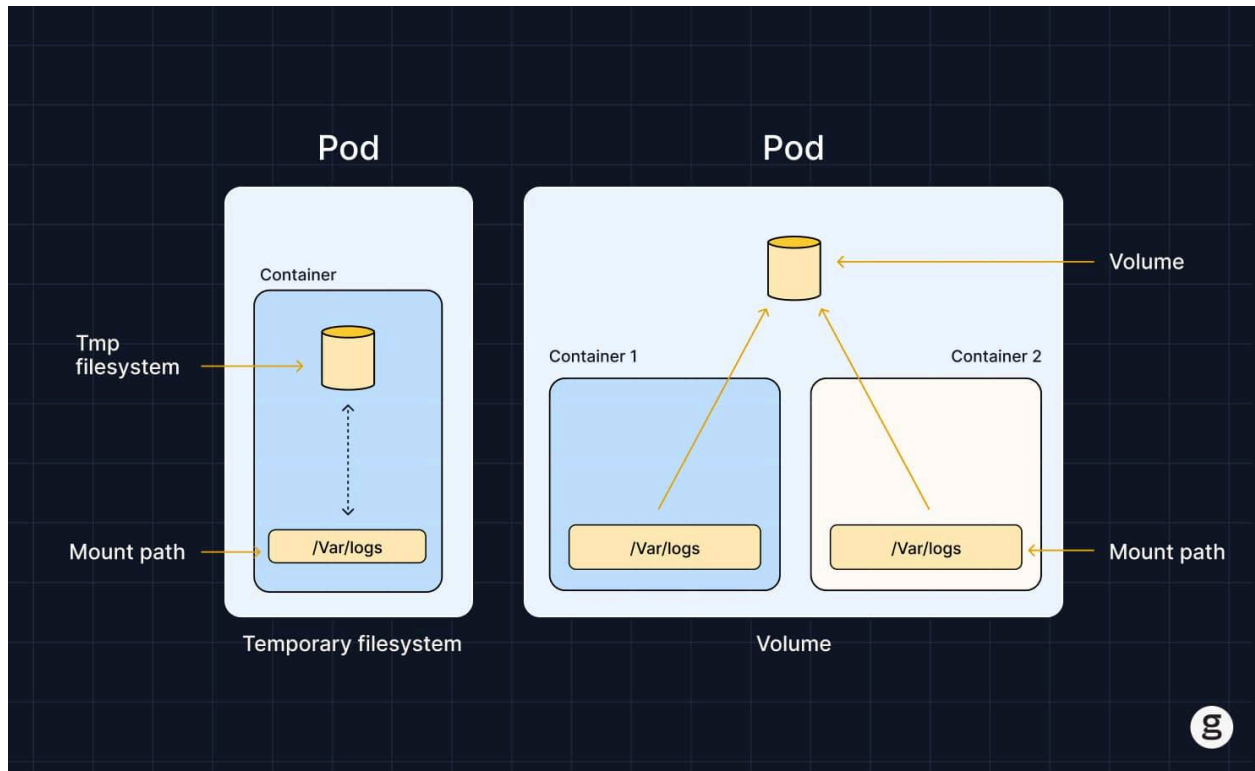
👉 Lesson: **emptyDir** survives container restart, NOT pod deletion.

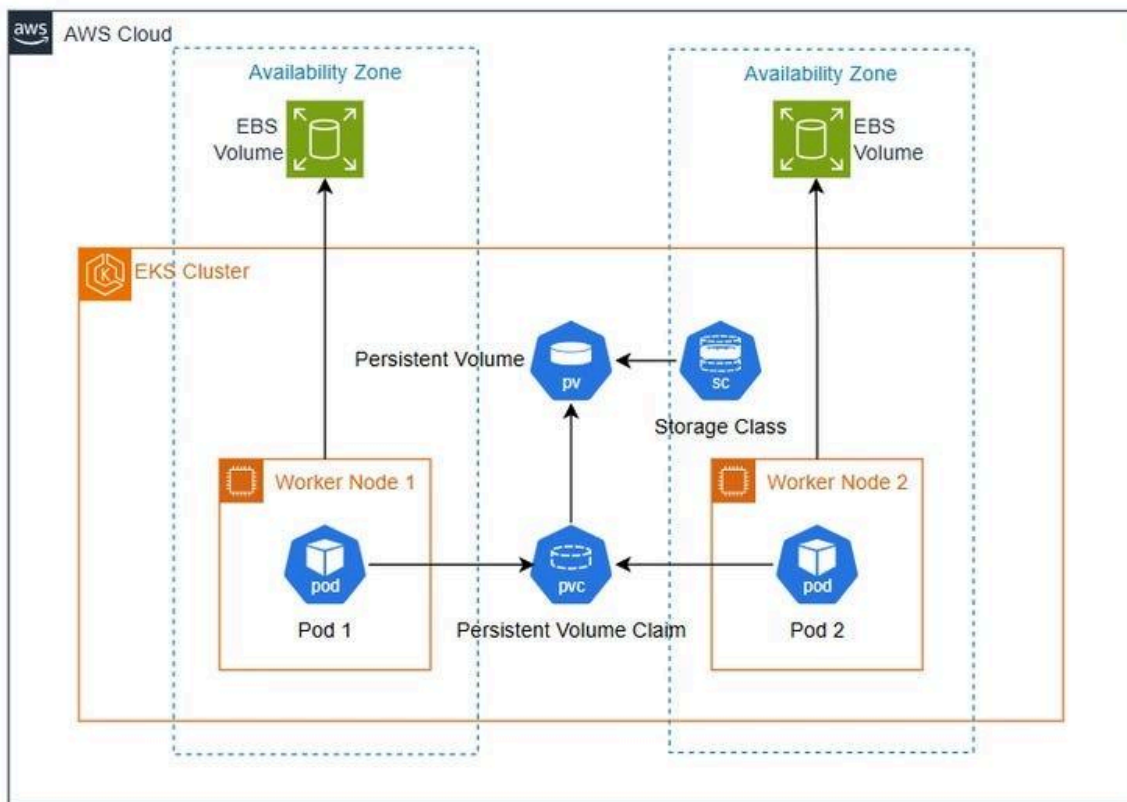
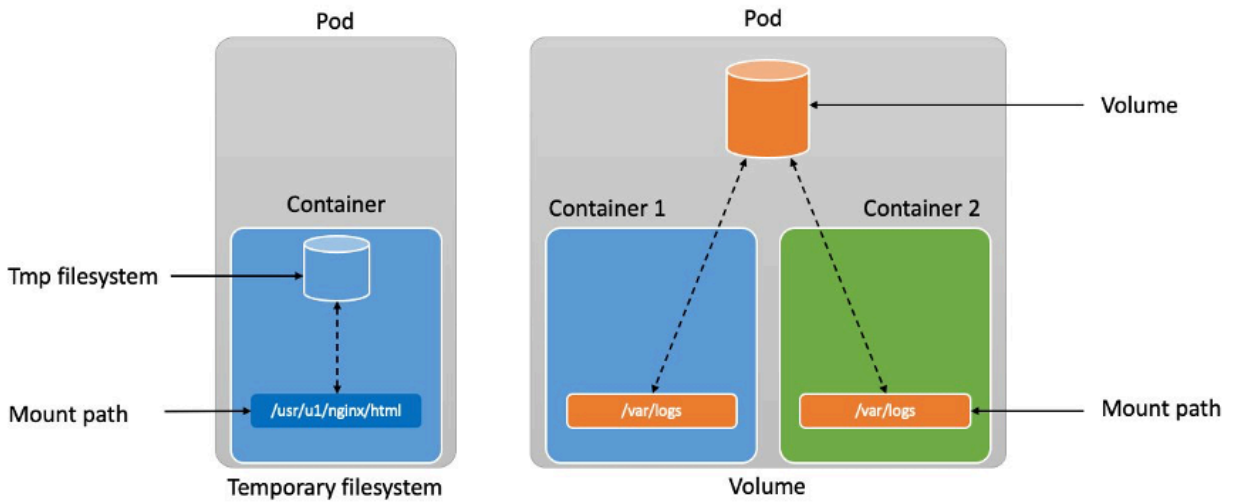
🔧 Lab 2 – Static Provisioning (Manual PV + PVC)

🎯 Goal

Understand how PVC binds to manually created PV.

Architecture





EKS Storage with Amazon EBS (Elastic Block Store)

Step 1 – Create EBS Volume (AWS Console or CLI)

Example:

```
aws ec2 create-volume \  
  --availability-zone us-east-1a \  
  --size 5 \  
  --volume-type gp3
```

Note the Volume ID.

Step 2 – Create PersistentVolume

```
apiVersion: v1  
kind: PersistentVolume  
metadata:  
  name: manual-ebs-pv  
spec:  
  capacity:  
    storage: 5Gi  
  accessModes:  
    - ReadWriteOnce  
  persistentVolumeReclaimPolicy: Retain  
  awsElasticBlockStore:  
    volumeID: <volume-id>  
    fsType: ext4
```

Apply:

```
kubectl apply -f pv.yaml  
kubectl get pv
```

Status → Available

Step 3 – Create PVC

apiVersion: v1

kind: PersistentVolumeClaim

metadata:

name: manual-pvc

spec:

accessModes:

- ReadWriteOnce

resources:

requests:

storage: 5Gi

Apply:

kubectl apply -f pvc.yaml

kubectl get pvc

Status → Bound

Step 4 – Use PVC in Pod

apiVersion: v1

kind: Pod

metadata:

name: static-app

spec:

containers:

- name: app

image: nginx

```
volumeMounts:
- mountPath: /data
  name: storage
volumes:
- name: storage
  persistentVolumeClaim:
    claimName: manual-pvc
```

Test:

```
kubectl exec -it static-app -- bash
echo "Persistent Data" > /data/file.txt
```

Delete pod → recreate → file still exists 

Step 5 – Delete PVC

```
kubectl delete pvc manual-pvc
```

Check PV:

```
kubectl get pv
```

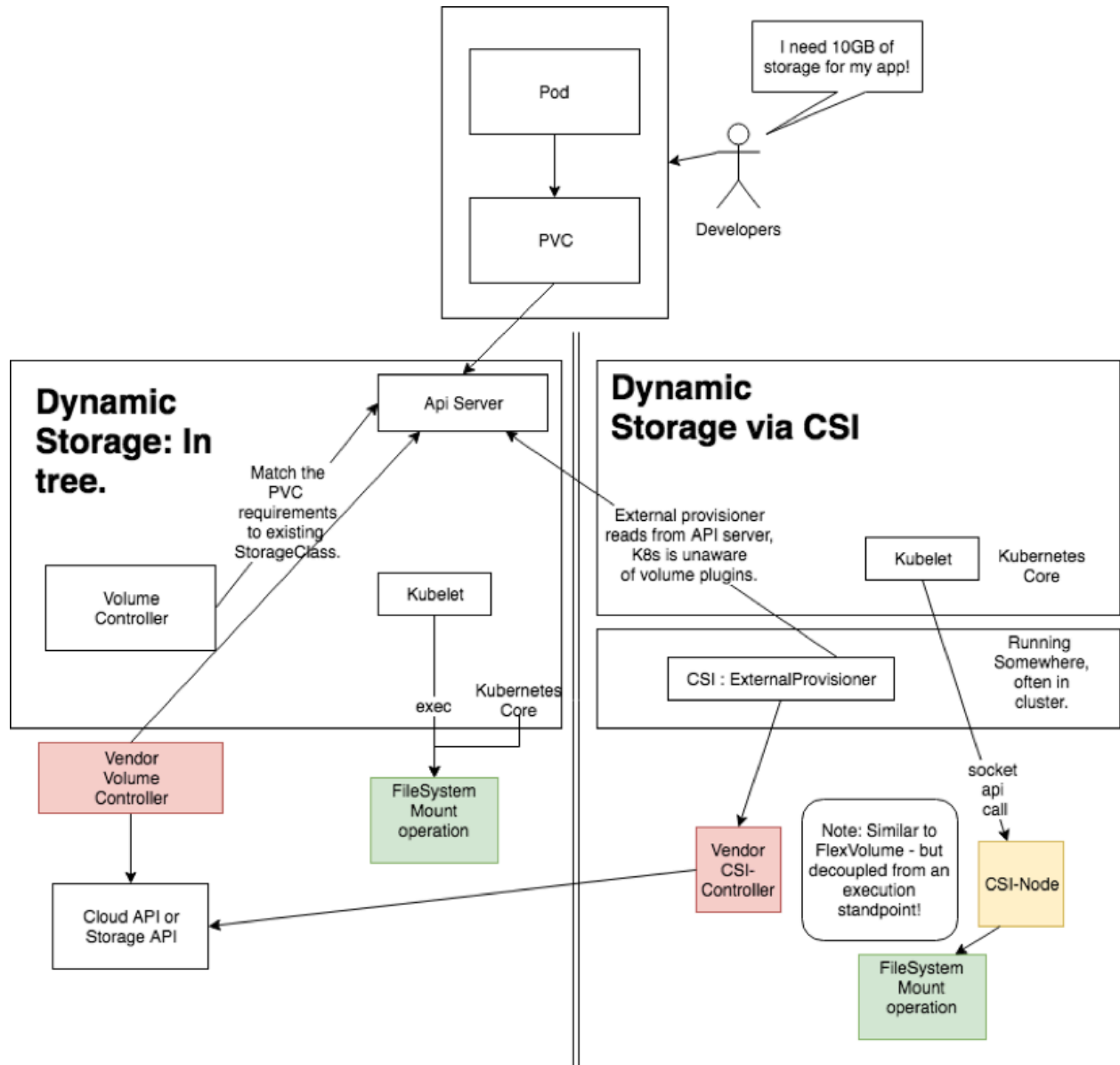
Status → Released (because reclaimPolicy: Retain)

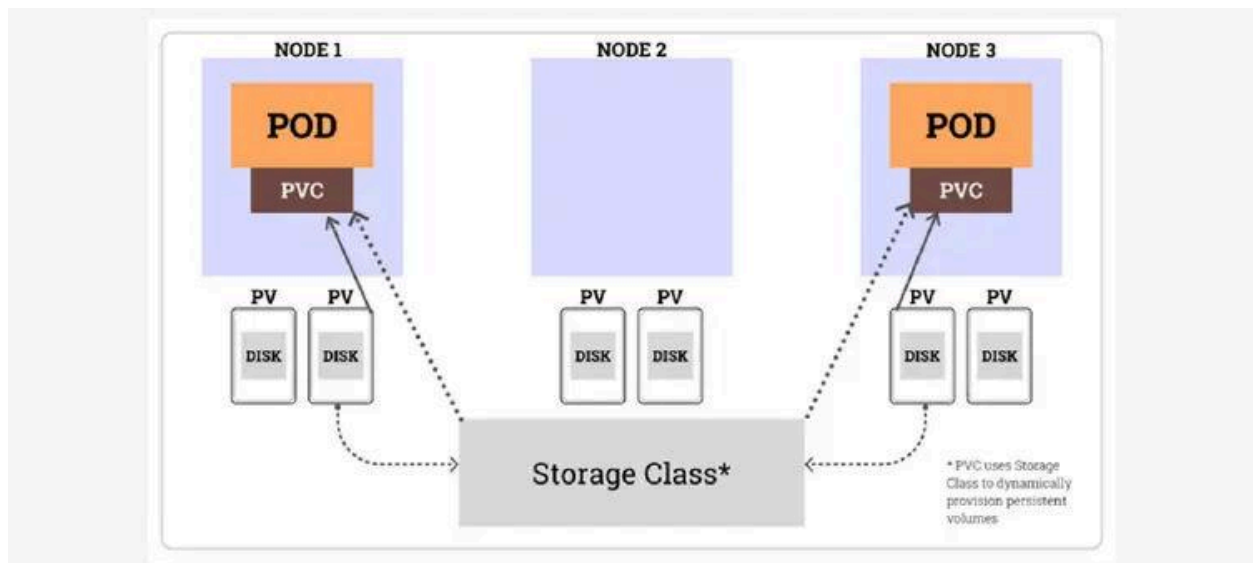
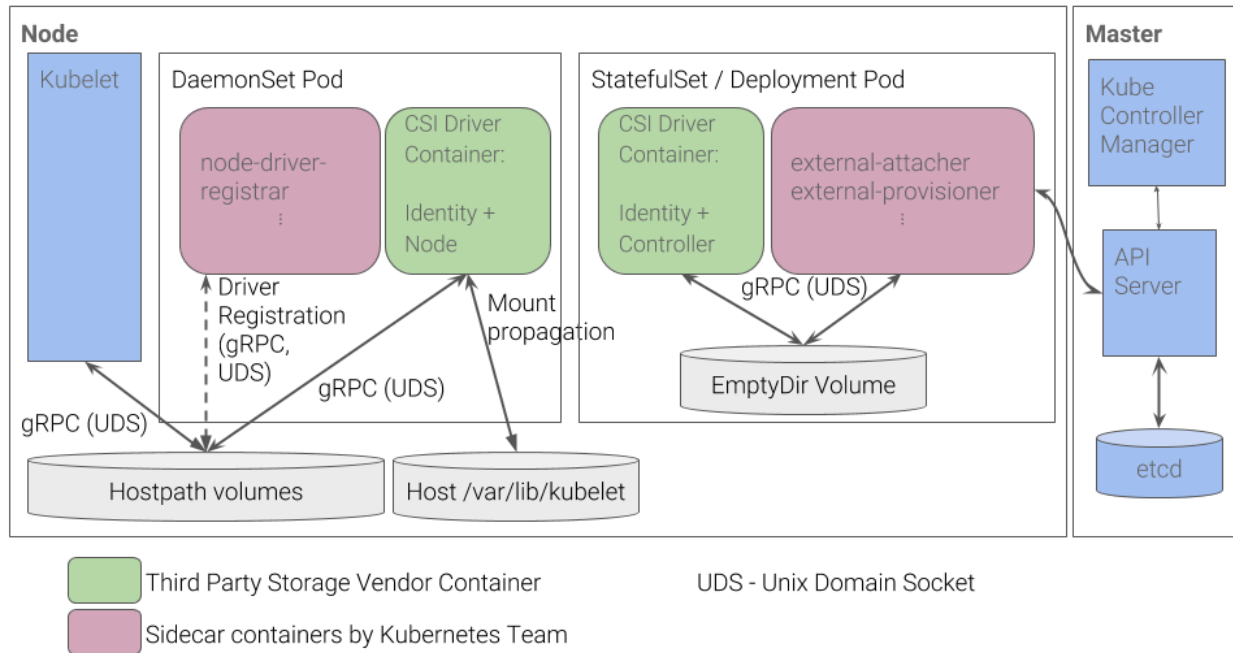
👉 Data still exists in AWS.

Lab 3 – Dynamic Provisioning (StorageClass)

This is what you'll use in production.

Architecture Overview





Step 1 – Check Existing StorageClass

```
kubectl get storageclass
```

Example output:

```
gp3 (default)
```

Step 2 – Create Custom StorageClass

```
apiVersion: storage.k8s.io/v1
```

```
kind: StorageClass
```

```
metadata:
```

```
  name: fast-storage
```

```
provisioner: ebs.csi.aws.com
```

```
parameters:
```

```
  type: gp3
```

```
reclaimPolicy: Delete
```

```
volumeBindingMode: WaitForFirstConsumer
```

Apply:

```
kubectl apply -f sc.yaml
```

Step 3 – Create PVC (No PV created manually!)

```
apiVersion: v1
```

```
kind: PersistentVolumeClaim
```

```
metadata:
```

```
  name: dynamic-pvc
```

```
spec:
```

```
  storageClassName: fast-storage
```

```
  accessModes:
```

- ReadWriteOnce

resources:

requests:

storage: 4Gi

Apply:

```
kubectl apply -f dynamic-pvc.yaml
```

Check:

```
kubectl get pvc
```

```
kubectl get pv
```

👉 Kubernetes automatically creates PV + EBS volume.

Step 4 – Use in Deployment

```
apiVersion: apps/v1
```

```
kind: Deployment
```

```
metadata:
```

```
  name: dynamic-app
```

```
spec:
```

```
  replicas: 1
```

```
  selector:
```

```
    matchLabels:
```

```
      app: demo
```

```
  template:
```

```
    metadata:
```

```
      labels:
```

```
        app: demo
```

```
    spec:
```

```
containers:
- name: app
  image: nginx
  volumeMounts:
  - mountPath: /data
    name: storage
volumes:
- name: storage
  persistentVolumeClaim:
    claimName: dynamic-pvc
```

Step 5 – Verify AWS Volume

kubectl describe pv

Check AWS Console → EBS volume automatically created.

Lab 4 – Reclaim Policy Behavior

Modify StorageClass:

reclaimPolicy: Retain

Delete PVC:

kubectl delete pvc dynamic-pvc

Observe:

- PV status: Released
- EBS volume still exists

Lab 5 – Volume Binding Mode

Immediate

Volume created immediately when PVC is created.

WaitForFirstConsumer (Recommended)

Volume created when Pod is scheduled → ensures correct AZ.

Test:

1. Create PVC
 2. Check PV → Not created
 3. Deploy Pod
 4. Now PV created
-

Lab 6 – StatefulSet with VolumeClaimTemplates

Used for databases.

apiVersion: apps/v1

kind: StatefulSet

metadata:

 name: web

spec:

 serviceName: "nginx"

 replicas: 2

 selector:

 matchLabels:

 app: nginx

 template:

```
metadata:
  labels:
    app: nginx
spec:
  containers:
  - name: nginx
    image: nginx
    volumeMounts:
    - name: www
      mountPath: /data
  volumeClaimTemplates:
  - metadata:
      name: www
    spec:
      accessModes: ["ReadWriteOnce"]
      storageClassName: fast-storage
      resources:
        requests:
          storage: 2Gi
```

Check:

```
kubectl get pvc
```

You'll see:

```
www-web-0
```

```
www-web-1
```

Each replica gets its own volume.

What You Learned

Concept	Real Understanding
emptyDir	Pod-level ephemeral storage
PV	Actual storage resource
PVC	Storage request
StorageClass	Enables dynamic provisioning
ReclaimPolicy	Controls deletion behavior
volumeBindingMode	Controls when volume is created
StatefulSet	Stateful apps get unique volumes

Production Best Practices (kops on AWS)

- Use **CSI driver (ebs.csi.aws.com)** (not in-tree driver)
 - Always use **WaitForFirstConsumer**
 - Use **Delete** for dev, **Retain** for prod DBs
 - Enable EBS encryption
 - Monitor IOPS & throughput
 - Use snapshots for backup
-

Cleanup Commands

```
kubectl delete deployment dynamic-app
kubectl delete pvc dynamic-pvc
kubectl delete sc fast-storage
kubectl delete pv manual-ebs-pv
```

Also delete unused EBS volumes in AWS.
